REQUIREMENTS NOT MET

N/A

PROBLEMS ENCOUNTERED

1. Using the wrong bit mask for the input bit.
2. Incorrect Interrupt Vector Configuration
   +

FUTURE WORK/APPLICATIONS

1. Ensure to find the right delay to minimize the switch debounce.
2. Ensure to enable all level of interruption .
3. Multiple interrupt vector by 2 to avoid address conflict

PRE-LAB EXERCISES

i. Assuming that no interrupt has been previously configured, devise and describe a generalized series of steps for configuring any interrupt within the ATxmega128A1U, i.e., not just an interrupt within the TC system.

1. we define the interrupt vector at the beginning of the program memory. .org TCC0_OVF_vect. And then place a rjmp to ISR.

2. Enable the interrupt at the peripheral source. This step generates an interrupt request for a specific event. TCC0_INTCTRLA. Set the bit like : TC_OVFINTLVL_LO_gc

3. Enable the interrupt at PMIC using PMIC_CTRL.

4. Enable Interrupts Globally using Sei

ii. Explain what happens in hardware (in other words, without the programmer's intervention) when the processor detects and then services (and returns from) an interrupt. Be as specific as possible, referencing certain registers when appropriate. You can assume that the reti instruction does not count as programmer intervention. You may provide a flowchart as a response, if desired.

When an enabled interrupt is triggered by a peripheral event,

1. hardware first finishes executing the current instruction.

2. It then automatically pushes the return address from the Program Counter (PC) onto the stack, decrementing the Stack Pointer (SP).

3. The hardware then loads the PC with the specific interrupt vector address, causing an immediate jump to the Interrupt Service Routine (ISR).

4. The Programmable Multilevel Interrupt Controller (PMIC) updates its STATUS register to block interrupts of the same or lower priority.

5. reti instruction at the end of the ISR, the hardware pops the original address from the stack back into the PC, increments the SP, and clears the PMIC's status flags, allowing the program to return where it left off.

PSEUDOCODE/FLOWCHARTS

Part 1

RESET:

set stack pointer

configure PORTD pin 0 as output

set TCC0 period = 18499

enable TCC0 overflow interrupt (low level)

enable PMIC low-level interrupts

enable global interrupts

start TCC0 with prescaler = 4

MAIN_LOOP:

repeat forever:

do nothing ; CPU is just looping

HARDWARE TIMER:

in parallel with CPU:

count clock cycles

when counter reaches PER:

reset counter to 0

trigger overflow interrupt

ON TCC0 OVERFLOW INTERRUPT:

CPU jumps to ISR

toggle PORTD pin 0

return from interrupt (resume MAIN_LOOP)

Part2 a

Define a register to be our counter, starting at 0. Define the locations for our Interrupt Service Routines in the Interrupt Vector Table.

Run these once at the start:

1. Initialize the SP so interrupts.

2. Set the counter to zero.

3. Configure Hardware Pins:

    Set all 8 pins of PORTC as outputs for the backpack LEDs.

    Turn them all off. Set pin PD6 as an output for the µPAD's blue LED. Turn it off.

    Set pin PF2 as an input for the S1 switch.

4. Configure the Interrupt for Switch S1 (PF2):

    Tell pin PF2 to enable its pull-up resistor and to trigger an interrupt on a falling edge (when the button is pressed).

    Tell Port F that pin PF2 is the specific pin that should be connected to the "Interrupt 0" line.

    Tell Port F to enable its "Interrupt 0" line and set its priority to "Low Level".

5. Globally Enable Interrupts:

    Tell the main interrupt controller (PMIC) to allow all "Low Level" interrupts.

    Tell the CPU to start listening for any and all enabled interrupts.

6. Jump to the Main Loop.

MAIN LOOP (Runs forever)

    1. Toggle the blue LED on pin PD6.

    2. rjmp main loop

IRS for S1 Press (Runs ONLY when S1 is pressed) :

    1. Increment the counter register.

    2. Take the new counter value, invert all its bits (for the active-low LEDs), and send it to PORTC to update the display.

    3. Clear the Port F interrupt flag so this interrupt doesn't immediately trigger again.

    4. Return to whatever the main loop was doing.

Part 2 b

START:

Initialize stack pointer

counter = 0

Configure PORTC as output for LEDs, turn them off

Configure PORTD pin 6 as output (blue LED), turn it off

Configure PORTF pin 2 (S1 switch) as input with pull-up

Configure S1 interrupt:

- Trigger on falling edge (button press)
- Enable interrupt on PF2, low priority

Configure debounce timer (TCD0):

- Period = 10 ms
- Overflow interrupt enabled, low priority

Enable low-level interrupts in PMIC

Enable global interrupts

MAIN LOOP (runs forever):

Toggle blue LED on PORTD pin 6 continuously

Repeat

IRS:

S1_PRESSED_ISR (triggered when button S1 pressed):

Disable S1 interrupt (to ignore bouncing)

Start debounce timer with prescaler 64

Return from interrupt

DEBOUNCE_TIMER_ISR (runs when 10 ms timer expires):

Save registers/status flags

Stop debounce timer

If switch (PF2) is still pressed:

VALID_PRESS:

counter = counter + 1

Update PORTC LEDs with complement of counter value

Else:

(It was a bounce, do nothing)

RESET_SWITCH_INTERRUPT:

Clear S1 interrupt flag

Re-enable S1 interrupt for next press

Restore registers/status flags

Return from interrupt

PROGRAM CODE

Part1

```
;
; lab3_1.asm , section 17b
;Name: Peter Feng
; class#: 11303
; Pi's Name: Grant
;description: configure a t/C to trigger overflow interrupt every 37ms



.include "ATxmega128a1udef.inc"



.org 0x0000
     rjmp reset

.org TCC0_OVF_vect
     rjmp TCC0_OVF_ISR



.org 0x0100
reset:
     ldi r16, low(RAMEND)
     sts CPU_SPL, r16
     ldi r16, high(RAMEND)
     sts CPU_SPH, r16

     ; initailize the output pins
     ldi r16, 0x01
```

```
	sts PORTC_DIRSET, r16



	;Configure Timer C0 for 37ms overflow
	; PER = (output delay * clock rate )/ prescaler
	; PER = (0.037s * 2000000Hz ) / prescaler
	; to make PER under 2^16 which is 65536
	; try prescaler 2
	; 74000/2 = 37000 < 65536
	ldi r16, high(37000)
	sts TCC0_PER+1, r16
	ldi r16, low(37000)
	sts TCC0_PER, r16

	; enable all 3 level of interrupt

	; telling peripheral deveice that it can send a low level interrupt
	; overflow/interrupt   L0 means low priority , gc is predefind value that i
can use
	ldi r16, TC_OVFINTLVL_LO_gc      ; This is 0x01
	sts TCC0_INTCTRLA, r16 ; feed it through interrupt control

	; The PMIC level, the gatekeeper
	;
                ;Low level enable
	ldi r16, PMIC_LoLVLEN_bm  ; load 1
	sts PMIC_CTRL, r16     ; to enable interrupt being process

	; Global Master Switch
	sei  ; enable global interrupt
```

```
      ; start Timer with prescaler  = 2
      ldi r16, 0x02
      sts TCC0_CTRLA, r16

main_loop:
      ; it does nothing but wait for the interrupt to happend
      rjmp main_loop

;timer overflow ISR
TCC0_OVF_ISR: ; excute when timer overflow
      ;toggle portc output pin
      ldi r16, 0x01
      sts PORTC_OUTTGL, r16
      reti ; return the prorgam and wait for more interrupt
```

Part 2a

```
;
;lab3_2.asm , section 17b
;Name: Peter Feng
; class#: 11303
; Pi Name: Grant
;description: configure an I/O port interrupt so that responses to a tactile
switch on the OOTB
;SLB can be made asynchronously.



.include "ATxmega128a1udef.inc"


;defined the counter register to keep track of the time that pass
.def counter = r16


.org 0x0000
     rjmp RESET_INIT ; reset the interrupt


;PortF2 is s1
; Set the vector for Port F's Interrupt 0
.org PORTF_INT0_vect
    rjmp    S1_PRESSED_ISR


RESET_INIT:
      ; --- Initialize Stack Pointer ---
    ldi     r17, high(RAMEND)
    sts     CPU_SPH, r17
    ldi     r17, low(RAMEND)
    sts     CPU_SPL, r17
```

```
  ; initailze the counter to 0
clr     counter

  ;set PORTC as outputs
  ldi r17, 0xFF
  sts PORTC_DIR, r17
  sts PORTC_OUT, r17 ; turn off all LEDs

  ;set PORTD, pin 6 which is BLUE_PWM LED as output
  ldi r17, 0b01000000
  sts PORTD_DIRSET, r17

  ; initally turn it off, since it's active low, write 1 into it
  sts PORTD_OUTSET, r17

  ; Set PORTF2 (S1) into input
  ldi r17, 0b00000100
  sts PORTF_DIRCLR, r17


  ;Configure Interrupt for S1
; Configure PF2 to trigger on a (button press) falling edge
  ; to select "Pull-up (on input)", the OPC bits [5:3] must be 011
  ; to "Sense falling edge", the ISC bits [2:0] must be 010
  ; PORT_OPC_PULLUP_gc = 0x18, PORT_ISC_FALLING_gc = 0x02.
  ;                         bitwise OR to combine 2 together
ldi     r17, PORT_OPC_PULLUP_gc | PORT_ISC_FALLING_gc
sts     PORTF_PIN2CTRL, r17
```

```
      ; decide which pin on Port F
      ;are triggering the Interrupt 0 signal
      ldi r17, 0b00000100
      ; Port F's Interrupt 0, only use Pin 2.
      sts PORTF_INT0MASK, r17


                ;gc apply to interrupt 0 , set prority to low
      ldi r17, PORT_INT0LVL_LO_gc
      ;eable interrupt at the source
      sts PORTF_INTCTRL, r17



    ; Enable low-level interrupts in the PMIC
    ldi     r17, PMIC_LOLVLEN_bm ; PMIC_LOLVLEN_bm = 0x01. Low Level Interrupt
Enable bit.
    sts     PMIC_CTRL, r17


    ; Enable global interrupts
    sei


;this is the background task that continously running
MAIN_LOOP:
      ldi r17, 0b01000000 ; pin6
      sts PORTD_OUTTGL, r17 ; toggle the blue LED
      rjmp MAIN_LOOP              ; without delay


S1_PRESSED_ISR:
      ;since the main is actively using the r17 to toggle the blue LED
      ; ISR also needs r17 for its own caculation
```

```
     ; we need to save a copy of what's in r17 onto stack
     push r17
     in  r17, CPU_SREG ; this copy all status register like the zero  flags
                              ; and save a copy on the stack
     push r17



     inc counter ; increment the counter

     ; LED is active low, that's why we complement it
     mov r17, counter ; copy what's in counter into r17
     com r17 ; complement r17 , 0 to 1, 1 to 0
     sts PORTC_OUT, r17 ; update the LED with this count

     ; this will clear the interrupt flag, so this ISR doesn't triger again
     ldi     r17, PORT_INT0IF_bm
    sts     PORTF_INTFLAGS, r17

     pop     r17       ;get the last saved copy of SREG—from the stack and puts it
back into r17.
    out     CPU_SREG, r17
    pop     r17



     ;Return from Interrupt back to main loop where it left off
     reti
```

```
Part2b
; ;
;lab3_2b.asm , section 17b
;Name: Peter Feng
; class#: 11303
; Pi Name: Grant
;description: Debounce s1 interrupt


.include "ATxmega128a1udef.inc"


;defined the counter register to keep track of the time that pass
.def counter = r16
; --- MODIFICATION ---
; Defined r17 as temp for clarity, as it is used throughout
.def temp = r17

.org 0x0000
    rjmp RESET_INIT ; reset the interrupt

;PortF2 is s1
; Set the vector for Port F's Interrupt 0
.org (PORTF_INT0_vect * 2)   ; * 2 to give .org its byte address
    rjmp    S1_PRESSED_ISR


; Added the interrupt vector for the debounce timer : TCD0
.org (TCD0_OVF_vect * 2)     ;* 2 to give .org its byte address
```

```
    rjmp    DEBOUNCE_TIMER_ISR



RESET_INIT:
     ; --- Initialize Stack Pointer ---
    ldi     r17, high(RAMEND)
    sts     CPU_SPH, r17
    ldi     r17, low(RAMEND)
    sts     CPU_SPL, r17

    ; initailze the counter to 0
    clr     counter

    ;set PORTC as outputs
    ldi r17, 0xFF
    sts PORTC_DIR, r17
    sts PORTC_OUT, r17 ; turn off all LEDs

    ;set PORTD, pin 6 which is BLUE_PWM LED as output
    ldi r17, 0b01000000
    sts PORTD_DIRSET, r17

    ; initally turn it off, since it's active low, write 1 into it
    sts PORTD_OUTSET, r17

    ; Set PORTF2 (S1) into input
    ldi r17, 0b00000100
    sts PORTF_DIRCLR, r17
```

```
;Configure Interrupt for S1
; Configure PF2 to trigger on a (button press) falling edge
; to select "Pull-up (on input)", the OPC bits [5:3] must be 011
; to "Sense falling edge", the ISC bits [2:0] must be 010
; PORT_OPC_PULLUP_gc = 0x18, PORT_ISC_FALLING_gc = 0x02.
;                          bitwise OR to combine 2 together
ldi     r17, PORT_OPC_PULLUP_gc | PORT_ISC_FALLING_gc
sts     PORTF_PIN2CTRL, r17

; decide which pin on Port F
;are triggering the Interrupt 0 signal



; Corrected the bitmask. 0b00000100 is for Pin 2.
ldi r17, 0b00000100
; Port F's Interrupt 0, only use Pin 2.
sts PORTF_INT0MASK, r17

        ;gc apply to interrupt 0 , set prority to low
ldi r17, PORT_INT0LVL_LO_gc
;eable interrupt at the source
sts PORTF_INTCTRL, r17



; Add configuration for the debounce timer (TCD0).
; We set it up for a 10ms delay.
; Calculation: PER = (Delay * Clock / Prescaler)
; PER = (0.010s * 2,000,000Hz / 64)  = 312
```

```
    ldi     r17, high(312)
    sts     TCD0_PER+1, r17
    ldi     r17, low(312)
    sts     TCD0_PER, r17

    ; Enable the overflow interrupt for debounce timer at a low level.
    ldi     temp, TC_OVFINTLVL_LO_gc
    sts     TCD0_INTCTRLA, temp



    ; Enable low-level interrupts in the PMIC
    ldi     r17, PMIC_LOLVLEN_bm ; PMIC_LOLVLEN_bm = 0x01. Low Level Interrupt
Enable bit.
    sts     PMIC_CTRL, r17

    ; Enable global interrupts
    sei

;this is the background task that continously running
MAIN_LOOP:
    ldi r17, 0b01000000 ; pin6
    sts PORTD_OUTTGL, r17 ; toggle the blue LED
    rjmp MAIN_LOOP          ; without delay



; This first ISR triggers immediately on a press to start the debounce timer.
S1_PRESSED_ISR:
    ; Disable the switch interrupt to ignore bounces
    ldi     r17, PORT_INT0LVL_OFF_gc
    sts     PORTF_INTCTRL, r17
```

```
    ; Start the 30ms debounce timer with a prescaler of 64
    ldi     r17, TC_CLKSEL_DIV64_gc
    sts     TCD0_CTRLA, r17

    reti

; This second ISR triggers ONLY after the 30ms debounce timer overflows.
DEBOUNCE_TIMER_ISR:
    ; since the main is actively using the r17 to toggle the blue LED
    ; ISR also needs r17 for its own caculation
    ; we need to save a copy of what's in r17 onto stack
    push    r17
    in      r17, CPU_SREG ; this copy all status register like the zero  flags
                          ; and save a copy on the stack
    push    r17

    ;Stop the debounce timer, it is no longer needed
    ldi     r17, TC_CLKSEL_OFF_gc
    sts     TCD0_CTRLA, r17

    ; Check if the switch is still pressed after the delay
    lds     r17, PORTF_IN
    sbrs    r17, 2             ; Skip the next instruction if bit 2 is 1
(released)
    rjmp    VALID_PRESS         ; If bit 2 is still 0 (pressed), it's a valid
press
```

```
    ; The PORTF2 is 1 the switch was released during the delay (it was a bounce)
pressed and quickly released.
    ; We jump directly to the reset routine without incrementing the counter.
    rjmp    RESET_SWITCH_INTERRUPT

VALID_PRESS:
    inc     counter ; increment the counter

    ; LED is active low, that's why we complement it
    mov     r17, counter ; copy what's in counter into r17
    com     r17 ; complement r17 , 0 to 1, 1 to 0
    sts     PORTC_OUT, r17 ; update the LED with this count

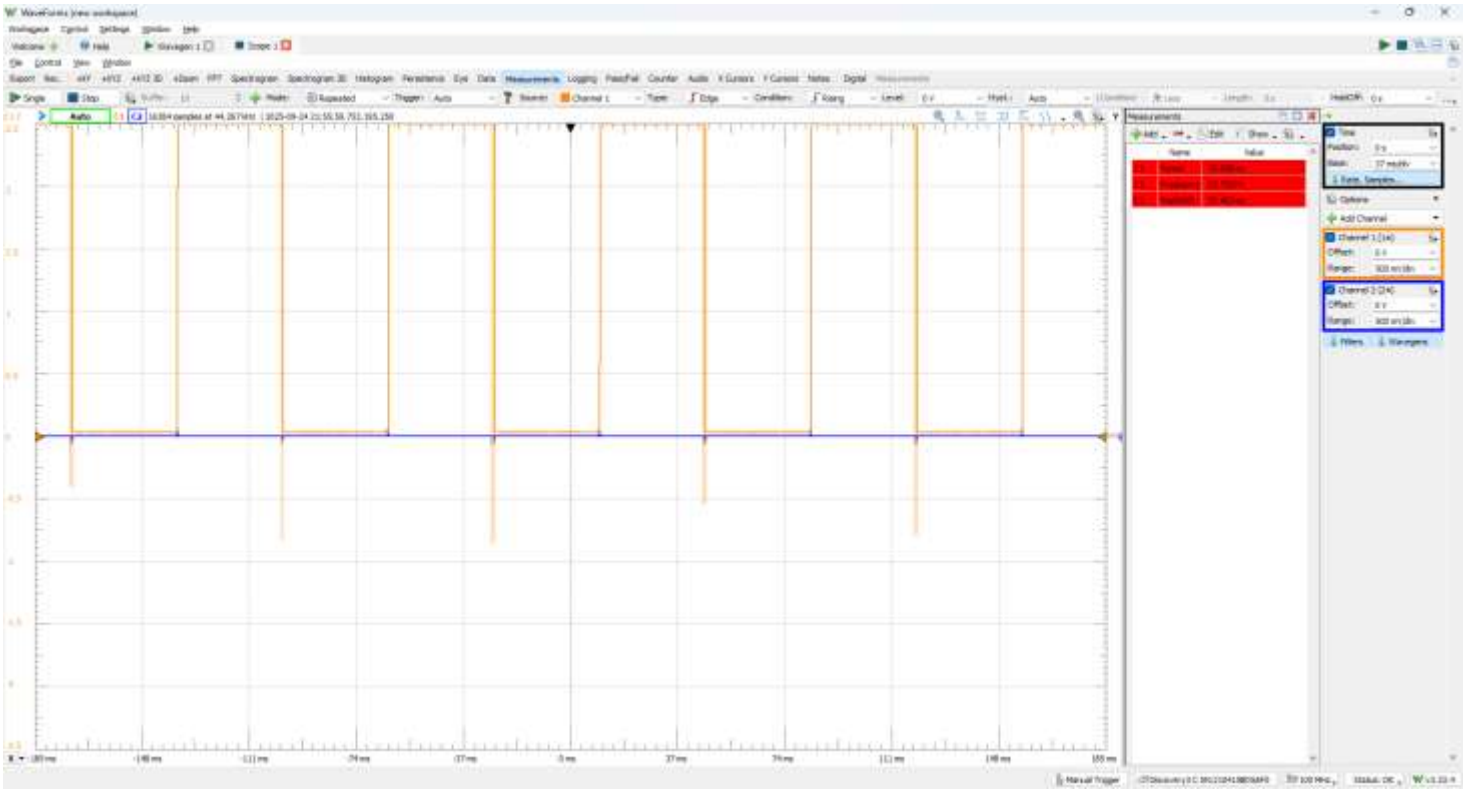

RESET_SWITCH_INTERRUPT:
    ; this will clear the interrupt flag, so this ISR doesn't triger again
    ldi     r17, PORT_INT0IF_bm
    sts     PORTF_INTFLAGS, r17

    ; Re-enable the switch interrupt for the next press
    ldi     r17, PORT_INT0LVL_LO_gc
    sts     PORTF_INTCTRL, r17

    pop     r17           ;get the last saved copy of SREG—from the stack and puts
it back into r17.
    out     CPU_SREG, r17
    pop     r17

    ;Return from Interrupt back to main loop where it left off
    reti
```

APPENDIX



Part 1 , 37ms delay trigger by interrupt.